🧩 Problem: Find the Largest Item in a List

You want to write a function to find the largest number in a list of integers.

rust Copy Edit

```rust
// Step 1: A function that works only with i32
fn largest_i32(list: &[i32]) -> &i32 {
    let mut largest = &list[0]; // assume first is largest

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];
    let result = largest_i32(&number_list);
    println!("The largest number is: {}", result);
}
```

But what if you want to find the largest **character** too?

rust Copy Edit

```rust
// Step 2: Duplicate function for chars
fn largest_char(list: &[char]) -> &char {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}
```

Now you have **code duplication**. Let's fix this using **generics**!

💡 Step 3: Use Generics in Functions

rust

Copy Edit

```rust
// Step 3: Combine both with generics
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    // T: PartialOrd means T must be comparable using >
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];
    let result = largest(&number_list);
    println!("Largest number is: {}", result);

    let char_list = vec!['y', 'm', 'a', 'q'];
    let result = largest(&char_list);
    println!("Largest char is: {}", result);
}
```

🧠 **Note:** `T: PartialOrd` ensures the type can be compared (like using `>`). Otherwise, Rust doesn't know if your custom type supports comparison.

🧱 Step 4: Use Generics in Structs

Let's define a `Point` struct for x and y values.

```rust
// Step 4: Generic Struct
struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let int_point = Point { x: 5, y: 10 }; // both i32
    let float_point = Point { x: 1.0, y: 4.0 }; // both f64
}
```

If you try different types for x and y:

```rust
// This will NOT compile
// let invalid_point = Point { x: 5, y: 4.0 };
```

That's because both fields must be the same type `T`.

🔄 Step 5: Use Multiple Generics in Structs

To allow different types for `x` and `y`, use two generic types.

rust

Copy Edit

```rust
// Step 5: Struct with different types
struct Point<T, U> {
    x: T,
    y: U,
}

fn main() {
    let both_int = Point { x: 5, y: 10 };
    let both_float = Point { x: 1.0, y: 4.0 };
    let mixed = Point { x: 5, y: 4.0 }; // Now valid
}
```

🔘 Step 6: Use Generics in Enums

Rust's standard library already does this.

```rust
// Step 6: Option Enum
enum Option<T> {
    Some(T),
    None,
}

// Result Enum (used for errors)
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Now you know why `Option<i32>` or `Option<String>` works — it's generic!

🛠️ Step 7: Implement Methods on Generic Structs

rust

Copy Edit

```rust
// Step 7: Method for all types
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let p = Point { x: 5, y: 10 };
    println!("p.x = {}", p.x());
}
```

rust

Copy Edit

```rust
// Step 7: Method for all types
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    // Method to access x
    fn x(&self) -> &T {
        &self.x
    }

    // Constructor method to make a new Point
    fn new(x: T, y: T) -> Self {
        Point { x, y }
    }
}

fn main() {
    let p = Point::new(5, 10); // use constructor
    println!("p.x = {}", p.x());
}
```

✅ Output:

ini

Copy Edit

```ini
p.x = 5
```

This `new` method makes it easier and cleaner to create `Point` objects from anywhere in your code. Let me know if you also want a version that supports different types like `Point<T, U>`.

✅ Full Example:

rust

Copy Edit

```rust
// Step 8: Method only for specific type (f32)
struct Point<T> {
    x: T,
    y: T,
}

// Method for all types
impl<T> Point<T> {
    fn new(x: T, y: T) -> Self {
        Point { x, y }
    }

    fn x(&self) -> &T {
        &self.x
    }
}

// Method only for f32
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}

fn main() {
    let p1 = Point::new(3.0f32, 4.0f32); // f32 point
    println!("Distance from origin: {}", p1.distance_from_origin());

    let p2 = Point::new(5, 10); // i32 point
    println!("p2.x = {}", p2.x());
    // p2.distance_from_origin(); ❌ ERROR: not available for i32
}
```

📌 Output:

csharp

Copy Edit

```csharp
Distance from origin: 5
p2.x = 5
```

✅ Updated Example with Simple Names

rust

Copy Edit

```rust
// Struct has generic types T and U
struct Point<T, U> {
    x: T,
    y: U,
}

// Method takes another Point with types X and Y
impl<T, U> Point<T, U> {
    fn mixup<X, Y>(self, other: Point<X, Y>) -> Point<T, Y> {
        Point {
            x: self.x,   // Take x from self (type T)
            y: other.y,  // Take y from other (type Y)
        }
    }
}

fn main() {
    let p1 = Point { x: 5, y: 10.4 };           // Point<i32, f64>
    let p2 = Point { x: "Hello", y: 'c' };      // Point<&str, char>

    let p3 = p1.mixup(p2);                      // Result: Point<i32, char>
    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
}
```

📌 Output:

ini

Copy Edit

```ini
p3.x = 5, p3.y = c
```

⚙️ Step 10: Performance of Generics (Monomorphization)

Rust compiles generic code into real, concrete code using **monomorphization**.

rust Copy Edit

```rust
// Example usage
let integer = Some(5);     // Option<i32>
let float = Some(5.0);     // Option<f64>
```

Behind the scenes, Rust turns this into:

rust Copy Edit

```rust
// Internally Rust creates:
enum Option_i32 {
    Some(i32),
    None,
}

enum Option_f64 {
    Some(f64),
    None,
}
```

✅ So there's **no performance cost** — it's like writing separate versions for each type!